



Processos, Threads, Escalonamento e Inferência Local com Ollama

Relatório Técnico — Atividade 1 (AV1) · Sistemas Operacionais 2026.2

Trilha A — Chat local: Ollama + Open WebUI

EQUIPE LLAMA

Carlos Henrico Fontes Cabral · Daniel de Andrade Lima · Lauren Victoria Ferreira Roma

Marcos Vinícius Assunção Nery Santos · Roseane Resende Couto · Wanessa Silva Santos



O que este relatório investiga

Aplicações de IA generativa local mobilizam, em tempo real, processos, threads, memória, armazenamento e rede — não apenas um modelo estático. Este trabalho documenta a instalação e observação de um sistema de inferência local em ambiente Linux via WSL, usando a Trilha A: Open WebUI como interface e Ollama como runtime, executando o modelo Llama-3.2-3B.

OBJETIVOS ESPECÍFICOS

- Mapear a árvore de processos (PID/PPID) e as chamadas de sistema entre Open WebUI e Ollama
- Medir consumo de CPU e memória nas fases de inicialização, carregamento e inferência
- Analisar escalonamento e latência sob diferentes cenários de concorrência
- Verificar o impacto do tamanho do contexto de entrada nos recursos e no tempo de resposta

PERGUNTA NORTEADORA

Como a camada de aplicação, o modelo de linguagem, a quantidade de parâmetros, a quantização e a configuração de execução afetam processos, threads, uso de CPU, memória, armazenamento e responsividade de um sistema local de IA generativa?

Base: análise de árvore de processos, strace e experimentos comparativos de carga



Trilha e modelo selecionados

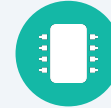


Trilha A — Ollama + Open WebUI

- Comunicação via API HTTP local, sem pipeline de RAG ou embeddings
- Maior facilidade de configuração e documentação mais ampla
- Foco total nos mecanismos de processos, threads e chamadas de sistema

172.26.19.219:11434 · porta 3000→8080

*Endereço do Ollama configurado manualmente no Open WebUI
(host.docker.internal não alcançou o WSL2)*



Llama-3.2-3B-Instruct-GGUF

3B

parâmetros

Q4_K_M

quantização 4-bit

128K

tokens de contexto

2.02 GB

tamanho do arquivo

Licença Llama 3.2 Community License · suporte oficial a português, inglês e outros 6 idiomas

Escolhido por equilibrar qualidade de resposta e requisito de hardware, respeitando o limite de 10B parâmetros da atividade.



Inventário de hardware e software

SISTEMA OPERACIONAL

Ubuntu 26.04.1 LTS (WSL2)

KERNEL

6.18.33.2-microsoft-standard-WSL2

PROCESSADOR

AMD Ryzen 5 6600H — 6 núcleos / 12 threads

MEMÓRIA RAM

8 GB host → 3,4 GiB visíveis ao WSL2

GPU

NVIDIA RTX 3050 — 4096 MiB VRAM (passthrough)

ARMAZENAMENTO

22 GB livres reais no C:\ do host (96% ocupado)

OLLAMA

versão 0.33.2

CAMADA DE APLICAÇÃO

Python 3.11.16 · Open WebUI (Docker)



Do navegador ao modelo, passo a passo



O frontend mantém a conexão via SSE (Server-Sent Events): cada token gerado pelo Ollama percorre esse caminho e chega ao navegador em tempo real, palavra por palavra.



Desafio de rede resolvido

host.docker.internal não alcançava o Ollama: Docker Desktop e a distro WSL2 operam em namespaces de rede distintos. Solução: configurar manualmente o IP da interface WSL2 (172.26.19.219:11434) no Open WebUI.



Quem faz o trabalho pesado?

Processo	PID	PPID	Threads	%CPU	%MEM
ollama (daemon)	148	1	18	0,4%	0,6%
llama-server (inferência)	1135	148	24	92,5%	26,2%
open-webui (uvicorn)	427	404	—	—	—

Snapshot via `ps -eo pid,ppid,stat,nlwp,pcpu,pmem,comm · pstree -p`



Memória mapeada: 51,6 GB virtuais vs. apenas 1,0 GB residente por thread. O modelo é acessado via mmap direto do disco — os tensores GGUF são carregados sob demanda, sem duplicar dados na RAM.

24

threads no llama-server, paralelizando os cálculos de inferência

92,5%

da CPU concentrados no processo filho — o daemon ollama fica ocioso



O que o strace revela

76,5%

do tempo total registrado pelo
strace foi gasto em futex

*5.407 chamadas registradas —
sincronização e espera entre as 24
threads do llama-server, não cálculo
puro*

`clone / clone3`

30 chamadas

Criação do processo llama-server e de suas 24 threads

`accept4 / poll`

2.117 chamadas

Gestão de conexões HTTP e streaming das respostas via rede

`mmap`

453 chamadas

Mapeia o arquivo do modelo direto na memória, sem cópia integral



Três configurações experimentais

01

Padrão (GPU)

Execução nativa na RTX 3050. 3 prompts curtos + 3 longos, mais 1 execução de carga fria.

02

Concorrência

1 requisição isolada vs. 3 simultâneas, testando se o Ollama enfileira ou paraleliza de fato.

03

CPU Forçada

CUDA_VISIBLE_DEVICES=-1: mesmos prompts curtos, sem aceleração gráfica, para medir o custo de não usar GPU.

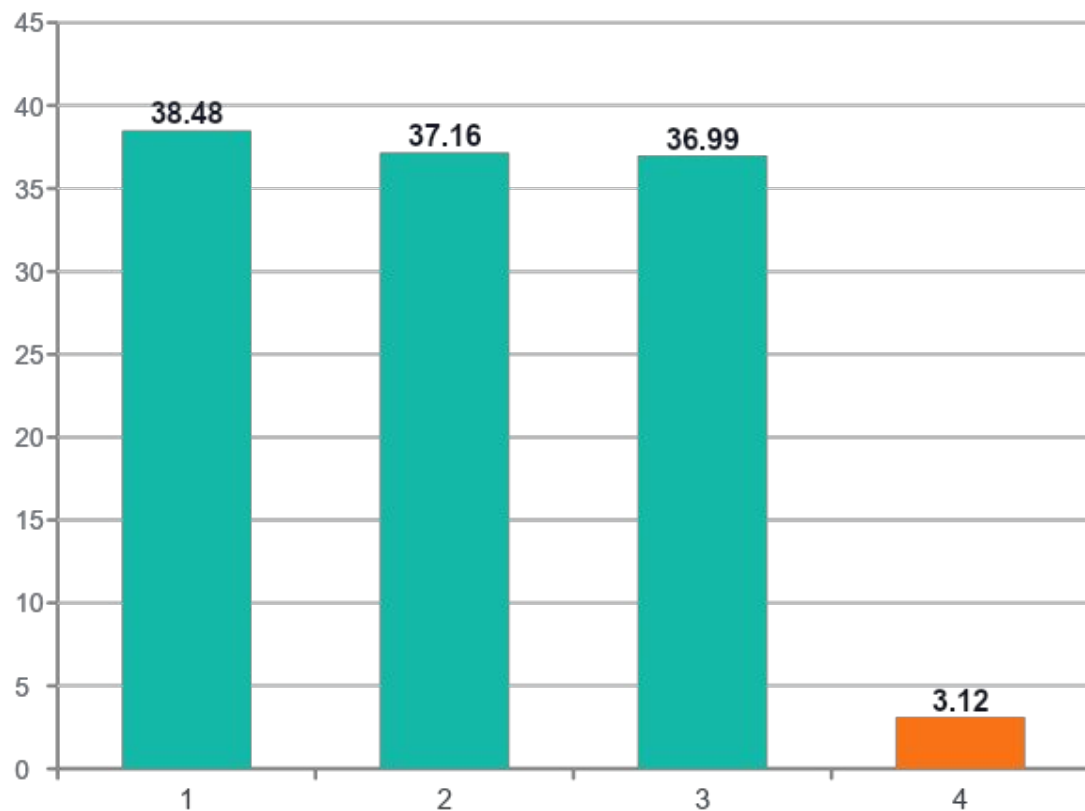
MÍNIMO DE 3 REPETIÇÕES POR CONFIGURAÇÃO

12 execuções mensuráveis no total, cobrindo entradas curtas (~17 tokens) e longas (~84 tokens)

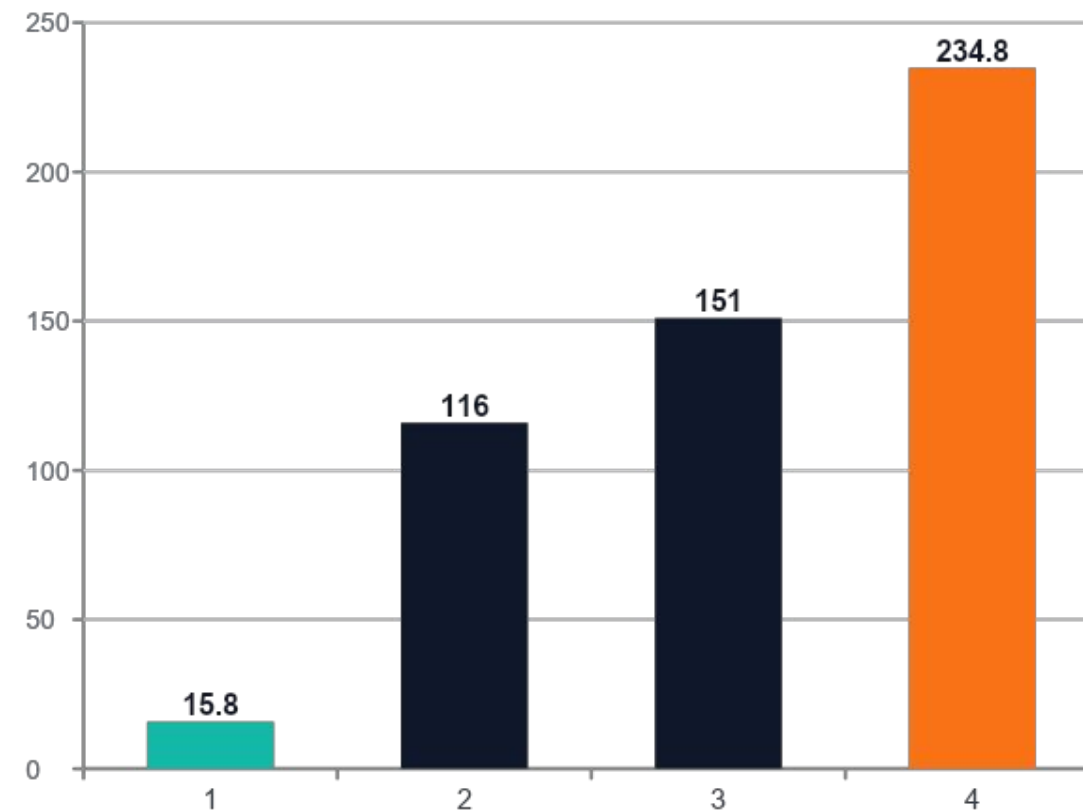


Vazão e latência por configuração

Vazão média de geração (tokens/s)



Tempo total da requisição HTTP (segundos)





Achados-chave

01

Sem paralelismo real

As 3 requisições simultâneas da Config. 2 foram enfileiradas (FIFO) pelo Ollama, não processadas ao mesmo tempo — a 3ª esperou ~139s na fila.

02

Swap derruba a vazão

782 MB de swap sob carga elevada reduziram a geração de ~27 para 1 token/s e causaram um timeout HTTP 500.

03

O trabalho pesado é do llama-server

92,5% de CPU e 24 threads concentrados no processo filho, enquanto o daemon ollama permanece ocioso (<1% CPU).

04

Tamanho do prompt não muda a velocidade por token

Prompts longos demoram mais no total (até 25s) mas mantêm tokens/s próximos aos prompts curtos.



Relação com Sistemas Operacionais



Processos & Threads

PID/PPID mapeados via pstree; llama-server soma 24 threads ativas



Escalonamento

O SO distribui ciclos de CPU entre as threads concorrentes de inferência



Gerência de Memória

mmap carrega tensores sob demanda; swap degrada a performance sob pressão



Chamadas de Sistema

futex, clone, accept4/poll e mmap revelam o comportamento interno via strace



E/S e Armazenamento

Leitura do arquivo do modelo (2,02 GB) e volumes/logs persistentes



Comunicação em Rede

Portas 11434 (Ollama) e 3000→8080 (Open WebUI) via sockets HTTP/SSE



Limitações do experimento



Monitoramento incompleto na concorrência

O monitor.sh só foi iniciado às 19:29, após as 3 requisições simultâneas (18:51–18:54) — sem dados de CPU/RAM desse intervalo.



Processo filho não monitorado em uma configuração

Métricas do daemon ollama (~0% CPU) não refletem o consumo real do llama-server, responsável pela inferência.



Pressão de memória sob carga alta

782 MB de swap comprometeram a estabilidade em uma requisição concorrente, gerando timeout HTTP 500.



CONCLUSÃO

A responsividade é ditada pela gestão de recursos do SO

O daemon ollama atua como escuta e gerenciador, enquanto o processo filho llama-server assume toda a carga computacional. O escalonador ajustou-se ativamente ao hardware disponível: 24 threads sob aceleração gráfica, apenas 6 threads físicas via OpenMP sob restrição de CPU.

O formato GGUF quantizado (Q4_K_M) aliado ao mmap viabilizou o sistema com apenas 4 GB de VRAM e 3,4 GB de RAM. Sem VRAM suficiente para paralelismo real, a concorrência esbarrou em fila FIFO e swap — evidenciando os limites do hardware, não do software.

TRABALHOS FUTUROS

Modelos menores (1B) ou quantização mais agressiva (Q2/Q3) · replicação em Linux nativo, sem WSL2

36–39 tok/s

vazão sob GPU (RTX 3050)

2–5 tok/s

vazão sob CPU forçada (580% CPU)

25x

queda de desempenho causada por swap



Referências

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg. *Operating System Concepts*. 10. ed. Hoboken: Wiley, 2018.

TANENBAUM, Andrew S.; BOS, Herbert. *Modern Operating Systems*. 5. ed. Boston: Pearson, 2023.

KERRISK, Michael. *The Linux Programming Interface*. San Francisco: No Starch Press, 2010.

LINUX MAN-PAGES PROJECT. *fork(2), execve(2), wait(2), clone(2), pthreads(7), sched(7)*.

Disponível em:

<https://man7.org/linux/man-pages/>

THE LINUX KERNEL DOCUMENTATION. *Scheduler documentation*. Disponível em:

<https://docs.kernel.org/scheduler/>

OLLAMA. *Repositório oficial*. Disponível em:

<https://github.com/ollama/ollama>

OLLAMA. *Documentação da API*. Disponível em:

<https://docs.ollama.com/api/introduction>

OPEN WEBUI. *Repositório oficial*. Disponível em:

<https://github.com/open-webui/open-webui>

OPEN WEBUI. *Documentação oficial*. Disponível em:

<https://docs.openwebui.com/>

HUGGING FACE. *Models*. Disponível em:

<https://huggingface.co/models>

NIST. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1, 2024. Disponível em:

<https://doi.org/10.6028/NIST.AI.600-1>

IBM. *O que é computação paralela?* IBM Think, 2025. Disponível em:

<https://www.ibm.com/br-pt/think/topics/parallel-computing>

Acesso em: 9 set. 2026.

SHOTTS, William. *The Linux Command Line*. LinuxCommand.org. Disponível em:

<https://linuxcommand.org/tlcl.php>